

Arquitetura do Sistema

Por que escolhemos este modelo

Monitoramento ambiental urbano com estações de baixo custo

Microserviços · MQTT · RabbitMQ · PostgreSQL · Redis

Resumo. O Pulso Urbano recebe continuamente as leituras de estações meteorológicas de baixo custo, transforma essas leituras em dados confiáveis e os entrega a gestores públicos e pesquisadores em painéis, relatórios e alertas automáticos. Este documento explica a arquitetura adotada pelo squad e, principalmente, o **porquê** de cada escolha: quais necessidades do cliente ela atende e quais vantagens ela traz para o produto hoje e nas próximas sprints.

Ideia central: nenhuma leitura de campo pode se perder. A arquitetura separa quem **recebe** o dado de quem o **processa**, guarda tudo primeiro e processa depois, e conecta as partes por mensagens. Assim, cada peça faz uma coisa só e pode evoluir sem derrubar as outras.

1. O problema que a arquitetura precisa resolver

Antes de escolher tecnologias, o squad levantou com o cliente as condições em que o sistema vai operar. Essas respostas definiram o desenho:

Necessidade levantada	O que isso exige da arquitetura
Recepção de dados é a função mais crítica	Se a ingestão para, a leitura daquele período some para sempre. A entrada de dados precisa ser simples, isolada e gravar antes de processar.
Uma leitura a cada 5 minutos por estação	Fluxo contínuo e previsível, que combina com mensageria assíncrona.
O sistema deve aceitar qualquer sensor	Regras de conversão configuráveis (fator e ganho) em vez de lógica embutida no hardware.
Retenção enquanto o projeto existir	Banco relacional robusto para série histórica de longo prazo.
Isolamento entre municípios do consórcio	Controle de acesso por perfil e por território aplicado em todos os serviços.
Alertas automáticos em condições críticas	Avaliação no momento em que o dado chega, sem depender de consultas periódicas.
50 usuários simultâneos, crescimento futuro	Partes que possam escalar de forma independente.

2. Visão geral

O diagrama abaixo organiza o sistema em seis blocos, da borda (estações) até o consumo (painéis e APIs). O caminho do dado é da esquerda para a direita.

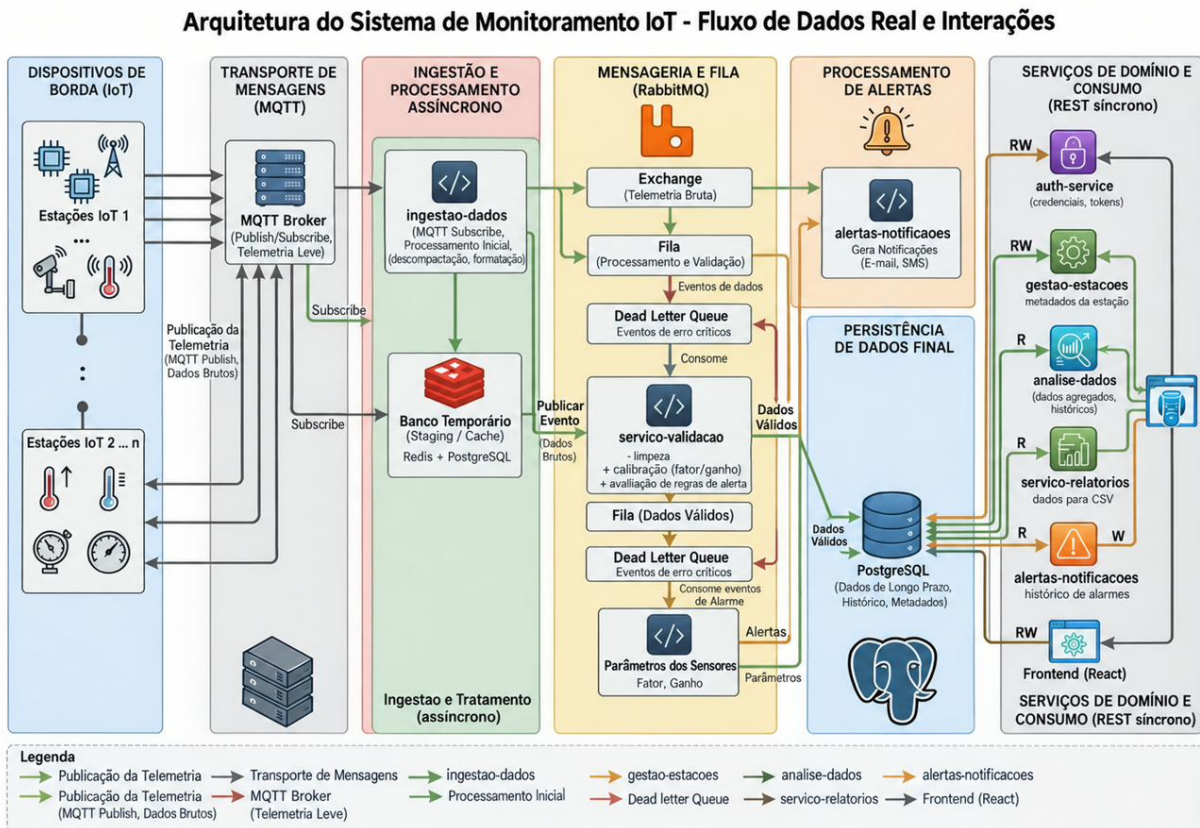


Figura 1 · Arquitetura do Pulso Urbano: fluxo de dados e interações entre os serviços.

Bloco	Papel
Dispositivos de borda (IoT)	Estações medem temperatura, umidade, PM2.5, vento, chuva e pressão e publicam a telemetria bruta.
Transporte (MQTT)	Broker leve que recebe as publicações das estações.
Ingestão e processamento assíncrono	<i>ingestao-dados</i> assina o MQTT e grava o dado bruto no banco temporário (Redis + PostgreSQL).
Mensageria e fila (RabbitMQ)	Distribui os eventos entre serviços, com filas de erro (Dead Letter Queue).
Processamento de alertas	<i>alertas-notificacoes</i> recebe os alarmes e envia os avisos.
Persistência e consumo	PostgreSQL guarda o histórico; serviços REST atendem o frontend React.

3. O caminho de uma leitura

Acompanhar uma única medição ajuda a entender por que cada peça está onde está:

1	Publicação A estação publica a leitura bruta via MQTT. Ela não carrega regra de negócio: só mede e envia.
2	Recepção O <i>ingestao-dados</i> assina o tópico, grava o payload bruto no banco temporário e publica um evento no RabbitMQ (exchange de telemetria bruta).
3	Validação e calibração O <i>servico-validacao</i> consome a fila, limpa o dado e aplica a calibração Valor_real = (Valor_bruto × Fator) + Ganho , com os parâmetros cadastrados para aquele sensor.
4	Persistência O valor calibrado é gravado no PostgreSQL e passa a fazer parte da série histórica.
5	Avaliação de alertas O mesmo serviço compara o valor com as regras ativas. Se houver violação, registra o alarme e publica o evento <i>alarme.disparado</i> .
6	Notificação O <i>alertas-notificacoes</i> consome o evento e dispara o aviso pelo canal configurado.
7	Consumo Painéis, relatórios e análises leem o PostgreSQL pelos serviços REST, sempre filtrando pelo município do usuário.

Repare que o dado é **gravado antes de ser processado** (passo 2). Se qualquer etapa seguinte precisar parar, a leitura continua guardada e é processada quando o serviço voltar.

4. Por que cada decisão

4.1 Microsserviços em vez de um único sistema

- Cada serviço tem uma responsabilidade clara: login, usuários, estações, ingestão, validação, análise, relatórios e alertas. Isso reduz o impacto de uma mudança e deixa o código mais fácil de entender.
- O ponto mais crítico, a recepção de dados, fica isolado. Um problema no painel ou nos relatórios não interrompe a ingestão.
- Serviços com carga diferente escalam separadamente: a ingestão pode ganhar mais instâncias sem mexer no restante.
- O trabalho do squad se divide naturalmente: cada integrante pode atuar em um serviço com pouca interferência nos demais.

4.2 MQTT na borda

- Protocolo feito para IoT: mensagens pequenas, baixo consumo de banda e de energia, ideal para estações de baixo custo.
- O modelo publicar/assinar desacopla a estação do servidor: ela só publica, sem saber quem vai consumir.
- As estações ficam "burras" de propósito: sem regra de negócio no hardware, qualquer ajuste é feito no servidor, sem atualizar firmware em campo.

4.3 Banco temporário (staging) antes do processamento

- Resposta direta à maior preocupação do cliente: o dado bruto é salvo no momento em que chega.
- Redis oferece escrita muito rápida como buffer; PostgreSQL garante durabilidade. As duas tecnologias ficam atrás da interface *StagingRepository*, o que permite ajustar o papel de cada uma sem alterar o restante do código.
- Guardar o dado bruto também permite reprocessar leituras caso uma calibração seja corrigida depois.

4.4 RabbitMQ entre os serviços, com Dead Letter Queue

- Comunicação assíncrona: quem publica não espera quem consome. Picos de dados viram fila, não erro.
- A Dead Letter Queue separa as mensagens que falharam, em vez de descartá-las. Elas podem ser analisadas e reprocessadas.
- Novos consumidores (por exemplo, um futuro serviço de estatística em tempo real) podem assinar os mesmos eventos sem alterar quem publica.

4.5 Calibração e alertas concentrados no serviço-validacao

- Uma única fonte de verdade para transformar dado bruto em dado real: a fórmula afim **(bruto × fator) + ganho** é aplicada em um só lugar.
- Os alertas são avaliados no mesmo ponto em que o dado calibrado nasce, então o alarme sempre usa o valor correto e é gerado no instante da leitura, sem varreduras periódicas no banco.
- Fator e ganho são parâmetros cadastrados por sensor. Isso atende o pedido de aceitar qualquer sensor: um sensor novo é uma configuração, não uma alteração de código.

4.6 Alertas orientados a eventos

- O *alertas-notificacoes* apenas reage ao evento *alarme.disparado*. Ele não precisa saber como o dado foi calibrado nem como a regra foi avaliada.
- Novos canais de aviso (e-mail, SMS e outros previstos para a Sprint 3) entram nesse serviço sem tocar no pipeline de dados.

4.7 PostgreSQL como persistência final

- Banco relacional maduro, adequado a séries históricas guardadas por tempo indeterminado, com índices e integridade referencial entre estações, sensores, medidas e alarmes.
- Uso da biblioteca *pg* sem ORM: SQL explícito, previsível e fácil de otimizar para as consultas agregadas dos painéis.
- O diagrama marca quem lê (R) e quem escreve (W/RW): os serviços de consumo, como análise e relatórios, apenas leem, o que protege a integridade do histórico.

4.8 Autenticação separada da gestão de usuários

- O *auth-service* só faz login e emite o token; o *gestao-usuarios* cuida de contas, papéis e vínculo municipal. Cada um muda por motivos diferentes, então faz sentido estarem separados.
- O token JWT carrega *id*, *papel* e *município*. Qualquer serviço valida o token sozinho, sem consultar o auth a cada requisição.
- Como o município vem do próprio token, o isolamento territorial é aplicado de forma uniforme: o Gestor Público só enxerga o próprio município e o Administrador tem visão global.

4.9 Sensores dentro do gestao-estacoes

- Sensor sempre pertence a uma estação. Mantê-los no mesmo serviço e no mesmo banco preserva a chave estrangeira *estacao_id* e evita um serviço extra sem ganho real.

4.10 Docker, Docker Compose e DevContainer

- Todo o ambiente (serviços, banco, filas, cache) sobe com a mesma configuração em qualquer máquina, o que facilita o trabalho do time e a transferência de tecnologia para o cliente.
- O mesmo empacotamento serve de base para o deploy em nuvem previsto para a Sprint 3.

5. Serviços e responsabilidades

Serviço	Responsabilidade	Acesso ao banco
auth-service	Login e emissão do token JWT	Leitura e escrita (credenciais)
gestao-usuarios	Contas, papéis e vínculo municipal	Leitura e escrita
gestao-estacoes	Estações, sensores (com fator e ganho) e regras de alerta	Leitura e escrita
ingestao-dados	Assinatura do MQTT e gravação no banco temporário	Staging (Redis + PostgreSQL)
servico-validacao	Limpeza, calibração, persistência e avaliação de alertas	Escrita de medidas e alarmes
analise-dados	Consultas agregadas e disponibilidade das estações	Somente leitura
servico-relatorios	Exportação em PDF/CSV	Somente leitura
alertas-notificacoes	Consumo de alarmes e envio de avisos	Histórico de alarmes

6. Qualidades que o modelo entrega

Atributo	Como a arquitetura atende
Resiliência	Gravação imediata no staging, filas persistentes e Dead Letter Queue: falhas de processamento não apagam leituras.
Escalabilidade	Serviços independentes e comunicação por fila. A carga do piloto (cerca de 0,08 mensagem por segundo com 4 estações) está muito abaixo da capacidade do modelo, que comporta centenas de estações.
Segurança	JWT com papel e município, senhas guardadas apenas como hash (bcrypt) e isolamento territorial em todas as rotas privadas.
Manutenibilidade	Responsabilidades pequenas e bem definidas, padrão de camadas (controllers, services, repositories) e SQL explícito.
Flexibilidade	Qualquer sensor entra por configuração de fator e ganho; novos consumidores assinam eventos já existentes.
Desempenho	Painéis leem dados já calibrados e agregados, sem recalcular a cada acesso.

Atributo	Como a arquitetura atende
Custo	Tecnologias abertas e estimativa de operação do piloto em torno de US\$ 25 a 30 por mês.

7. Preparada para as próximas sprints

A arquitetura já comporta as entregas planejadas sem mudanças estruturais:

Sprint	Entrega	Onde se encaixa
Sprint 2	Ingestão de ponta a ponta com simulação de várias estações	MQTT → ingestao-dados → RabbitMQ → servico-validacao, exatamente o fluxo do diagrama.
Sprint 2	Dashboards, relatórios e análise estatística	analise-dados e servico-relatorios lendo o histórico no PostgreSQL.
Sprint 3	Estação física e datalogger	A estação real publica no mesmo broker MQTT; nada muda do lado do servidor.
Sprint 3	Alertas com baixa latência e múltiplos canais	Novos canais entram no alertas-notificacoes, que já é orientado a eventos.
Sprint 3	Portal Público e comparação entre municípios	Novos consumidores de leitura sobre os mesmos dados, respeitando o isolamento por município.

8. Conclusão

O modelo foi escolhido porque responde diretamente às prioridades do cliente. A recepção de dados, apontada como a função mais crítica, fica isolada e grava tudo antes de processar. A calibração configurável atende o pedido de aceitar qualquer sensor. A mensageria garante que os alertas nasçam no momento da leitura, e o isolamento por município vem embutido no token usado por todos os serviços.

Além de resolver o problema de hoje, a arquitetura deixa o caminho aberto para a estação física, o portal público e novos canais de alerta, que entram como peças novas conectadas ao mesmo fluxo, sem reescrever o que já existe.

Em uma frase: estações simples na borda, dado guardado na chegada, processamento desacoplado por filas e serviços pequenos e focados, formando uma base confiável, segura e pronta para crescer.